Angular termék nyilvántartó

Beadandó

2026. 04. 07.

Készítette:Répási Levente Ádám

Szak:PTI

Neptunkód:
E9SHD7

Sárospatak, 2026

1. A projekt célja és áttekintése

A projekt célja egy egyszerű, böngészőalapú terméknyilvántartó rendszer megvalósítása, amellyel egy vállalat vagy kiskereskedő nyilvántarthatja a raktáron lévő termékeit. A rendszer lehetővé teszi termékek felvitelét, listázását és törlését, mindezt egy modern, reszponzív felületen keresztül.

A megvalósítás során a következő főbb szempontokat tartottam szem előtt:

- Egyszerű, átlátható felhasználói felület – minden funkcióhoz közvetlenül hozzá lehet férni.
- Biztonságos belépési mechanizmus – csak azonosítás után érhetők el az adatok.
- Adatok tartós tárolása – MongoDB adatbázis segítségével az adatok megmaradnak az alkalmazás újraindítása után is.
- Moduláris felépítés – a frontend és a backend külön fut, egymástól függetlenül fejleszthető.

A fejlesztés során Angular 17 keretrendszert, Node.js/Express backendet és MongoDB adatbázist alkalmaztam, ami egy elterjedt

2. Alkalmazott technológiák

Az alábbiakban összefoglalom a projekt elkészítéséhez felhasznált technológiákat és azok szerepét a rendszerben.

2.1 Frontend – Angular 17

Az Angular egy Google által fejlesztett, TypeScript alapú frontend keretrendszer. A projekt során az Angular 17-es verzióját alkalmaztam, amely bevezette az önálló (standalone) komponens modellt, ezáltal elkerülhető a hagyományos NgModule alapú architektúra.

Fontosabb Angular funkciók a projektben:

- Reaktív űrlapok (ReactiveFormsModule) – mezővalidáció közvetlen vezérlésére.
- Angular Router – oldalak közötti navigáció (bejelentkezés, terméklista, termék hozzáadása).
- HttpClient – REST API hívások kezelése.

2.2 Backend – Node.js + Express

A szerver oldal Node.js futtatókörnyezeten alapul, az Express keretrendszer segítségével. Az Express egy minimalista HTTP framework, amellyel gyorsan és rugalmasan definiálhatók az API végpontok. A backend kezeli a kérések fogadását, az adatok validálását és az adatbázis-kommunikációt.

2.3 Adatbázis - MongoDB + Mongoose

Az adatokat MongoDB dokumentumorientált adatbázisban tárolom, amelyet lokálisan futtatok (mongodb://localhost:27017/productdb). A Mongoose ODM (Object Document Mapper) réteg segítségével sémát definiáltam a termékekhez, így a bevitt adatok szerkezete és típusa ellenőrzött.

2.4 Bootstrap5

A felhasználói felület stílusát a Bootstrap 5 CSS keretrendszer adja, amely gyors, reszponzív elrendezést tesz lehetővé. Navigációs sáv, kártyák, táblázatok és gombok egyaránt Bootstrap osztályokkal vannak formázva.

3.A rendszer felépítése és architektúrája

A rendszer klasszikus kliens–szerver architektúrát követ. A frontend Angular alkalmazás a 4200-as porton fut, a backend REST API a 4000-es porton, az adatbázis pedig a 27017-es MongoDB alapértelmezett portján.

3.1 Könyvtárszerkezet

A project két fő részre tagolódik

Mappa / fájl	Leírás
product-manager-v2/	Gyökérkönyvtár (Angular projekt)
src/app/login/	Bejelentkezési komponens
src/app/product-list/	Terméklista komponens
src/app/add-product/	Termék hozzáadása komponens
src/app/app.service.ts	HTTP kommunikáció (API hívások)
src/app/app.routes.ts	Útvonalak definiálása
api/server.js	Express szerver belépési pont
api/routes/product.route.js	API végpontok
api/models/Product.js	Mongoose séma
api/database/db.js	Adatbázis kapcsolat beállítása

3.2 API végpontok

A backend három REST végpontot biztosít:

Végpont	Funkció
GET /api/getProducts	Összes termék lekérése
POST /api/addProduct	Új termék mentése az adatbázisba
GET /api/deleteProduct/:id	Termék törlése azonosító alapján

4. Funkciók részletes ismertetése

4.1 Bejelentkezés

Az alkalmazás megnyitásakor a felhasználó egy bejelentkezési képernyőt lát. Az adatokat reaktív Angular-form kezeli, kötelező mezővalidációval. A helyes felhasználónév és jelszó (admin / admin123) megadása után az alkalmazás a terméklistára navigál. Helytelen adatok esetén hibaüzenet jelenik meg.

A bejelentkezési logika frontend oldalon van implementálva; az adatok egy privát objektumban tárolódnak a LoginComponent-ben. Ez iskolai feladatra megfelelő, de éles környezetben backend hitelesítést kellene alkalmazni.

4.2 Terméklista

A terméklista oldal az összes adatbázisban tárolt terméket listázza egy táblázatban, amely a következő oszlopokat tartalmazza: sorszám, megnevezés, ár (HUF-ban), mennyiség (db), leírás és törlés gomb.

Az oldalon egy keresőmező is szerepel, amely valós időben szűri a termékeket név vagy leírás alapján. Ha nincs találat, tájékoztató üzenet jelenik meg. Ha az adatbázis nem elérhető, hibaüzenet tájékoztatja a felhasználót.

4.3 Termék hozzá adása

A termék felviteléhez egy külön oldal áll rendelkezésre. A következő adatokat lehet megadni:

- Megnevezés (kötelező, egyedi)
- Ár (kötelező, csak szám)
- Mennyiség (kötelező, csak szám)
- Leírás (opcionális, szabad szöveg)

Az űrlap csak akkor küldhető el, ha minden kötelező mező érvényes. Sikeres mentés után az alkalmazás 1,5 másodperccel visszanavigál a terméklistára. Ha már létezik azonos nevű termék, a rendszer figyelmeztetést jelenít meg.

4.4 Termék törlése

A terméklista minden sorában egy Törlés gomb szerepel. A gombra kattintva megerősítő párbeszédablak jelenik meg. Megerősítés esetén a rendszer elküldi a törlési kérést a backendnek, majd automatikusan frissíti a listát.

5. Adatmodell

Az adatbázisban egyetlen kollekció (products) található, amelybe a következő sémájú dokumentumok kerülnek:

Mezőnév	Típus / Megkötések
name	String – kötelező, egyedi
price	Number – kötelező, minimum 0
quantity	Number – kötelező, minimum 0
description	String – opcionális, alapértelmezetten üres
_id	ObjectId – MongoDB által automatikusan generált

A Mongoose séma a unique: true megkötéssel biztosítja, hogy két azonos nevű termék ne kerüljön az adatbázisba. Az adatbázis neve productdb, lokálisan fut a szabványos MongoDB porton.

5.1 Adatáramlás a rendszerben

A Mongoose séma a unique: true megkötéssel biztosítja, hogy két azonos nevű termék ne kerüljön az adatbázisba. Az adatbázis neve productdb, lokálisan fut a szabványos MongoDB porton.

6. Fejlesztés napló

Az alábbiakban összefoglalom a fejlesztés főbb lépéseit és a közbülső döntéseket.

Fázis	Elvégzett feladatok
1. Tervezés	Rendszerterv elkészítése, technológiai stack kiválasztása (Angular + Node + MongoDB). Az adatmodell megtervezése, API végpontok meghatározása.
2. Backend fejlesztés	Node.js/Express szerver felállítása, Mongoose séma (Product.js) elkészítése, három API végpont implementálása (lekérés, mentés, törlés), CORS beállítása.
3. Frontend – alapok	Angular projekt létrehozása, Bootstrap 5 integrálása, útvonalak (routes) definiálása, AppService megírása a HTTP hívásokhoz.
4. Komponensek	LoginComponent, ProductListComponent és AddProductComponent elkészítése. Reaktív form validáció mindkét adatbeviteli komponensnél.
5. Tesztelés	Manuális tesztelés: belépési hibakezelés, duplikált terméknév kezelése, üres lista megjelenítése, szűrési funkció ellenőrzése.
6. Javítások	Magyar változónevek és fájlnevek angolra cserélése (Termek.js → Product.js, termek-nyilvantarto → product-manager stb.) a tanári visszajelzés alapján.

7. Összefoglalás és értékelése

A terméknyilvántartó rendszer az összes kitűzött funkciót megvalósítja: a felhasználó be tud lépni, meg tudja tekinteni a terméklistát, tud szűrni, új terméket felvinni és meglévőt törölni. A rendszer Angular, Node.js és MongoDB technológiákra épül, ami modern, piacképes technológiai kombinációt jelent.

7.1 Továbbfejlesztési lehetőségek

A jelenlegi megvalósítás iskolai kereteken belül teljes körű, azonban éles környezetben az alábbi fejlesztések lennének indokoltak:

- Backend hitelesítés JWT token segítségével (jelenleg a login csak frontend oldalon működik).
- Termékszerkesztési lehetőség (PUT/PATCH végpont, szerkesztési form).
- Lapozás (pagination) nagy mennyiségű adat esetén.
- Törlés HTTP DELETE metódussal (jelenleg GET metódussal van megvalósítva).
- Unit tesztek Angular tesztelési keretrendszerrel (Jasmine/Karma).

7.2 Tanulság

A projekt segítségével mélyebb ismereteket szereztem az Angular komponens alapú architektúrájáról, a reaktív form validáció működéséről. A frontend és a backend külön futtatása jól szemlélteti a modern webfejlesztésben elterjedt szeparált kliens–szerver szemléletmódot.